

Optimization

Lecture 24

Dr. Colin Rundel

Optimization

Optimization problems underlie nearly everything we do in Machine Learning and Statistics.

Most models can be formulated as

$$P : \arg \min_{x \in D} f(x)$$

- Formulating a problem P is not the same as being able to solve P in practice
- Many different algorithms exist for optimization but their performance varies widely depending on the exact nature of the problem

The calculus approach

From calculus we know that to find the minimum of $f(x)$ we can set the derivative equal to zero and solve, $\nabla f(x) = 0$.

This *analytical* approach works well for simple, closed-form problems but breaks down quickly in practice:

- We are rarely optimizing simple functions of a single variable - most problems involve multiple parameters
- Many objective functions have no closed-form solution (e.g. logistic regression, lasso, neural networks)
- The system of equations may be too large or too complex to solve directly
- Even when a solution exists, finding *all* critical points and classifying them is often intractable

Instead we turn to **iterative numerical methods** that start from an initial guess and progressively improve it.

Gradients & Hessians

Gradient (first-order)

For a scalar-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the gradient is the vector of partial derivatives,

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

- Points in the direction of steepest *ascent* at the current point
- Its magnitude $\|\nabla f(x)\|$ gives the rate of steepest ascent
- At a (local) minimum, $\nabla f(x^*) = 0$

Gradient - examples

$$f(x, y) = x^2 + 3xy + y^2$$

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x + 3y \\ 3x + 2y \end{bmatrix}$$

$$f(x, y, z) = x^2 y + \sin(z)$$

$$\nabla f(x, y, z) = \begin{bmatrix} 2xy \\ x^2 \\ \cos(z) \end{bmatrix}$$

Hessian (second-order)

The **Hessian** is the $n \times n$ matrix of second partial derivatives,

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- Symmetric when f has continuous second partial derivatives
- Describes the local *curvature* of f - how the gradient changes

Hessian - examples

$$f(x, y) = x^2 + 3xy + y^2$$

$$\nabla^2 f(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$$

$$f(x, y, z) = x^2 y + \sin(z)$$

$$\nabla^2 f(x, y, z) = \begin{bmatrix} 2y & 2x & 0 \\ 2x & 0 & 0 \\ 0 & 0 & -\sin(z) \end{bmatrix}$$

Gradient Descent

Naive Gradient Descent

The basic idea behind this approach is that the gradient of a function tells us the direction of steepest ascent (or descent). Therefore, to find the minimum we should take our next step in the direction of the negative gradient to most quickly approach the nearest minima.

Given an n -dimensional function $f(x_1, \dots, x_n)$, and an current position x_k then our update rule is,

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

here α refers to the step length or the learning rate which determines how big a step we will take.

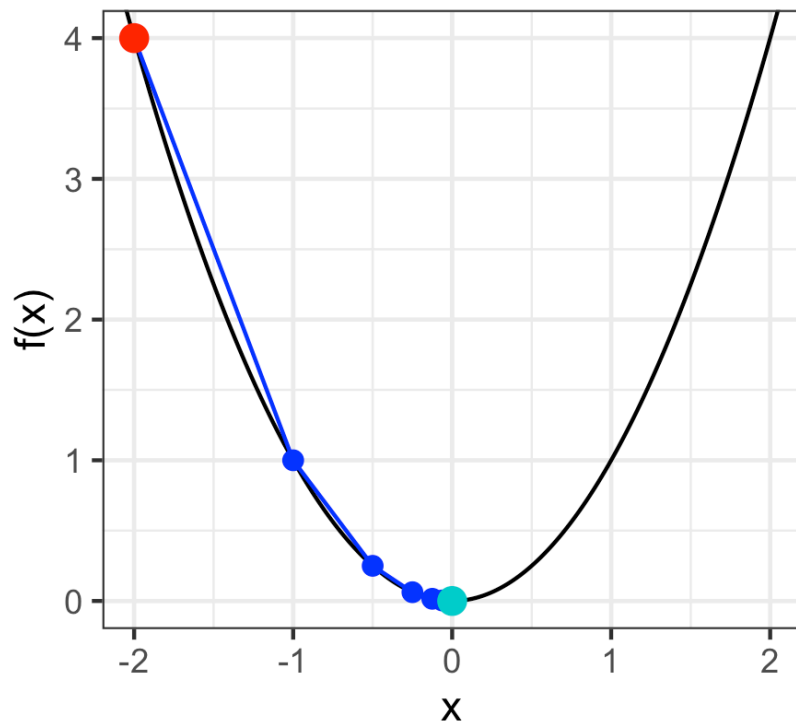
Implementation

```
1 grad_desc_1d = function(x, f, grad, step, max_step = 100, tol = 1e-6) {
2   res = list(x = x, f = f(x))
3
4   for (i in seq_len(max_step)) {
5     x_new = x - grad(x) * step
6
7     if (!is.finite(x_new) || !is.finite(f(x_new)))
8       break
9
10    if (abs(x_new - x) < tol)
11      break
12
13    x = x_new
14    res$x = c(res$x, x)
15    res$f = c(res$f, f(x))
16  }
17
18  if (i == max_step)
19    warning("Failed to converge!")
20
21  res
22 }
```

A basic example

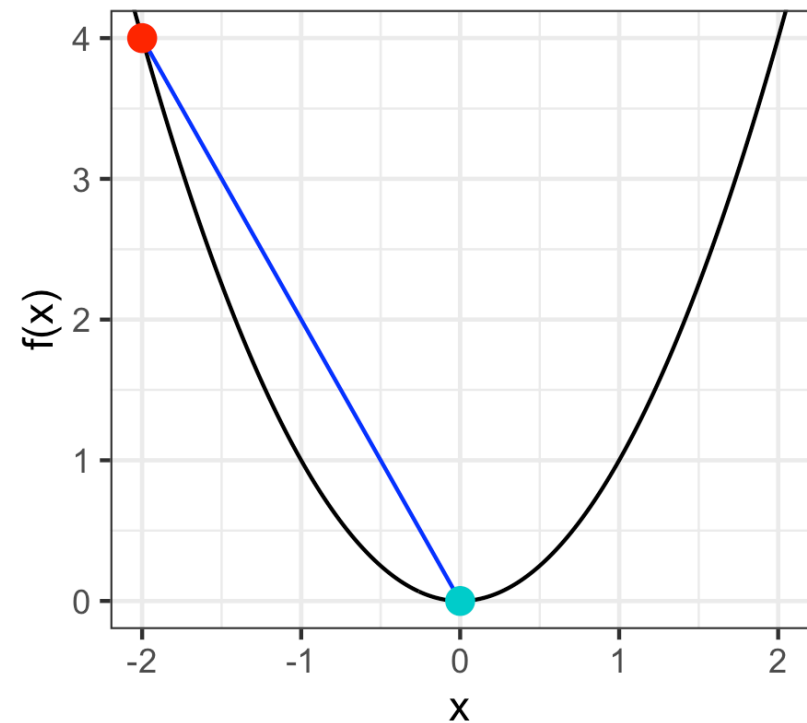
$$f(x) = x^2$$
$$\nabla f(x) = 2x$$

```
1 opt = grad_desc_1d(-2, f, grad, step = 0.25)
2 plot_1d_traj(c(-2, 2), f, opt)
```



```
1 f = \(x) x^2
2 grad = \(x) 2 * x
```

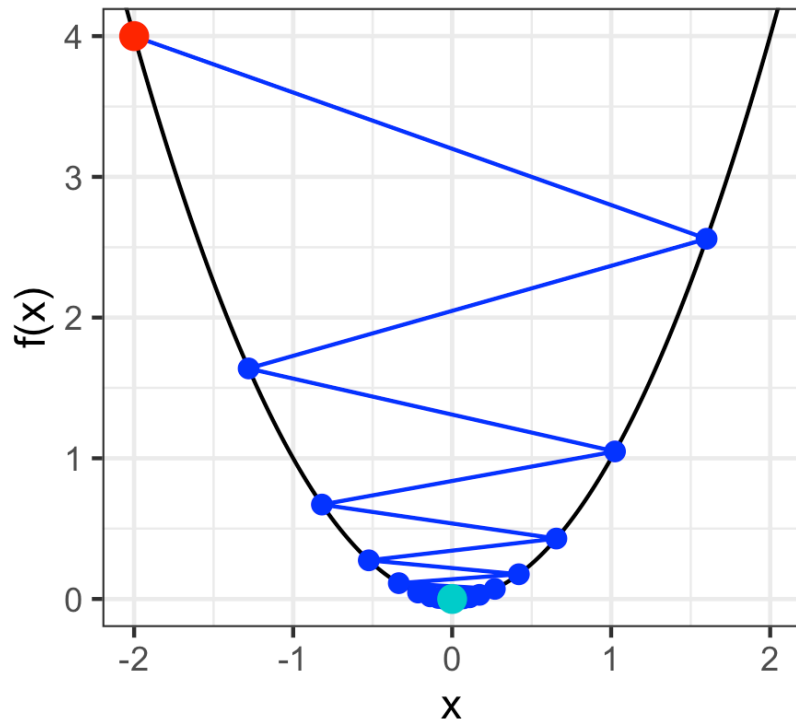
```
1 opt = grad_desc_1d(-2, f, grad, step = 0.5)
2 plot_1d_traj(c(-2, 2), f, opt)
```



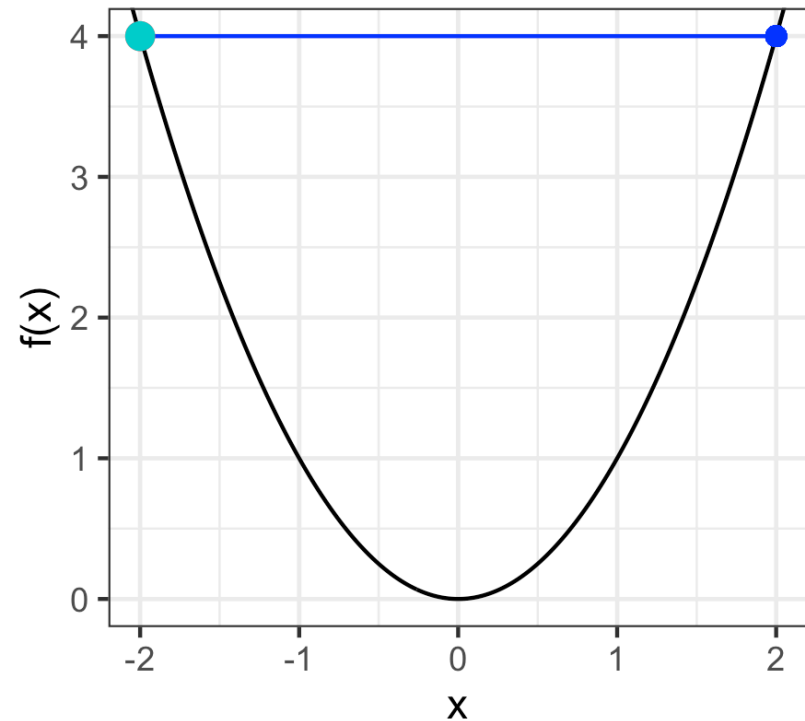
Where can it go wrong?

If you pick a bad step size ...

```
1 opt = grad_desc_1d(-2, f, grad, step = 0.9)
2 plot_1d_traj(c(-2, 2), f, opt)
```



```
1 opt = grad_desc_1d(-2, f, grad, step = 1)
2 plot_1d_traj(c(-2, 2), f, opt)
```



Local minima

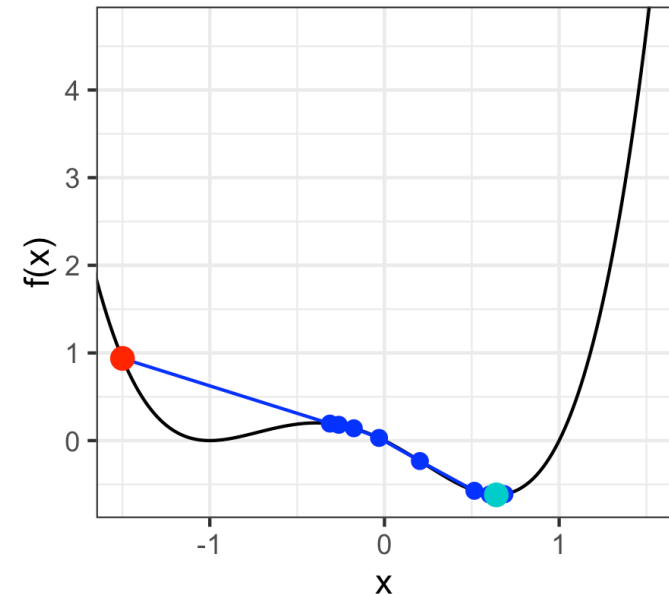
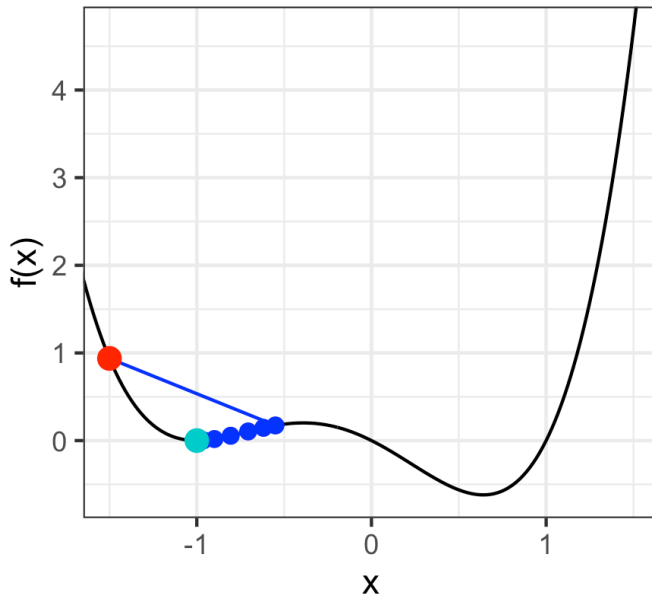
The function below has multiple minima, both starting point and step size affect the solution we obtain,

$$f(x) = x^4 + x^3 - x^2 - x$$
$$\nabla f(x) = 4x^3 + 3x^2 - 2x - 1$$

```
1 f = \ (x) x^4 + x^3 - x^2 - x
2 grad = \ (x) 4*x^3 + 3*x^2 - 2*x - 1
```

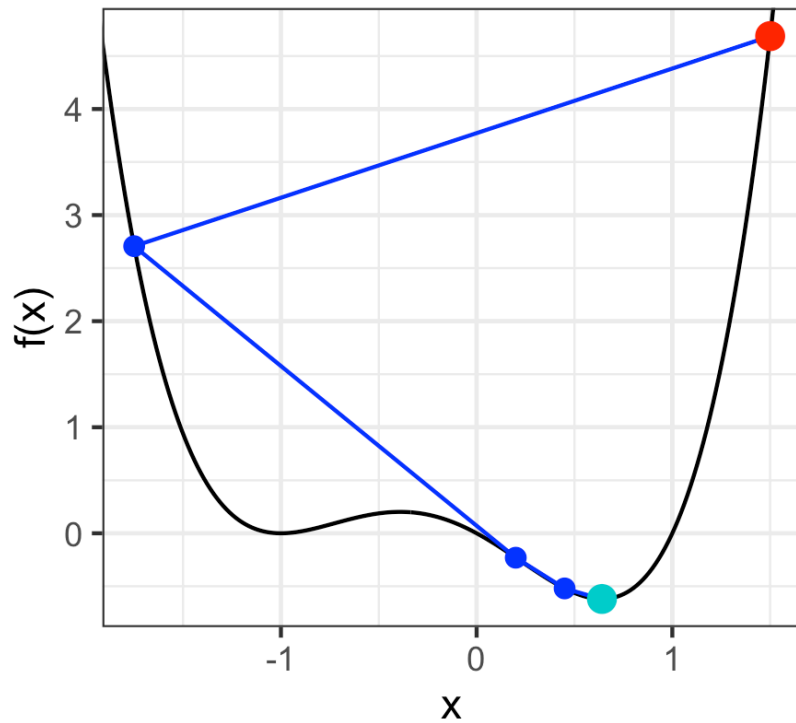
```
1 opt = grad_desc_1d(-1.5, f, grad, step = 0.1)
2 plot_1d_traj(c(-1.5, 1.5), f, opt)
```

```
1 opt = grad_desc_1d(-1.5, f, grad, step = 0.1)
2 plot_1d_traj(c(-1.5, 1.5), f, opt)
```

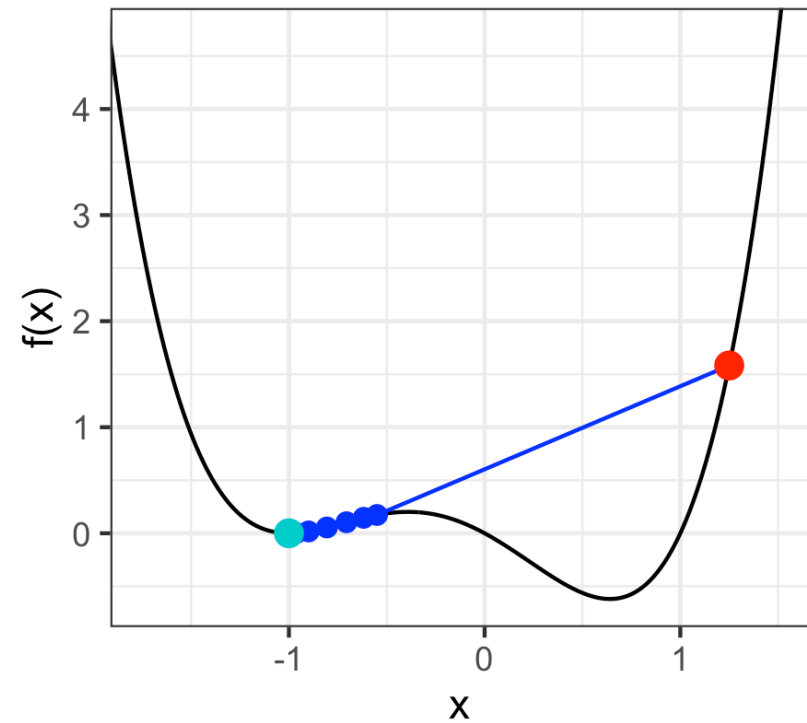


Alternative starting points

```
1 opt = grad_desc_1d(1.5, f, grad, step = 0.2)
2 plot_1d_traj(c(-1.75, 1.5), f, opt)
```



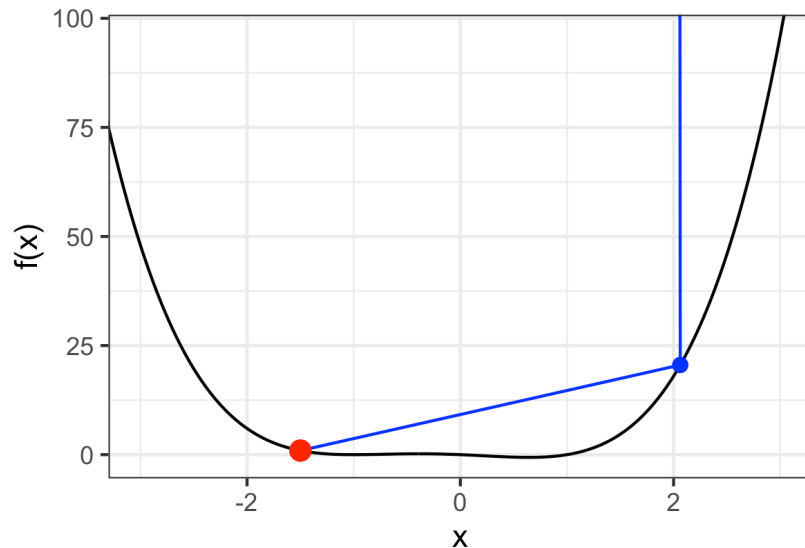
```
1 opt = grad_desc_1d(1.25, f, grad, step = 0.2)
2 plot_1d_traj(c(-1.75, 1.5), f, opt)
```



Problematic step sizes

If the step size is too large it is possible for the algorithm to overflow,

```
1 opt = grad_desc_1d(-1.5, f, grad, step = 0.1)
2 plot_1d_traj(c(-3, 3), f, opt)
```



```
1 opt$x
```

```
[1] -1.500000e+00  2.062500e+00 -2.998608e+01  7
[5] -1.467367e+15  9.478445e+45
```

```
1 opt$f
```

```
[1]  9.375000e-01  2.055299e+01  7.806665e+05  3
[5]  4.636118e+60  8.071392e+183
```

Gradient Descent w/ backtracking

As we have just seen having too large of a step can be problematic, one solution is to allow the step size to adapt.

Backtracking involves checking if the proposed move is advantageous, i.e.

$$f(x_k - \alpha \nabla f(x_k)) < f(x_k)$$

- If it is downhill then accept $x_{k+1} = x_k - \alpha \nabla f(x_k)$.
- If not, adjust α by a factor τ (e.g. 0.5) and check again.

Pick larger α to start (but not so large so as to overflow) and then let the backtracking tune things.

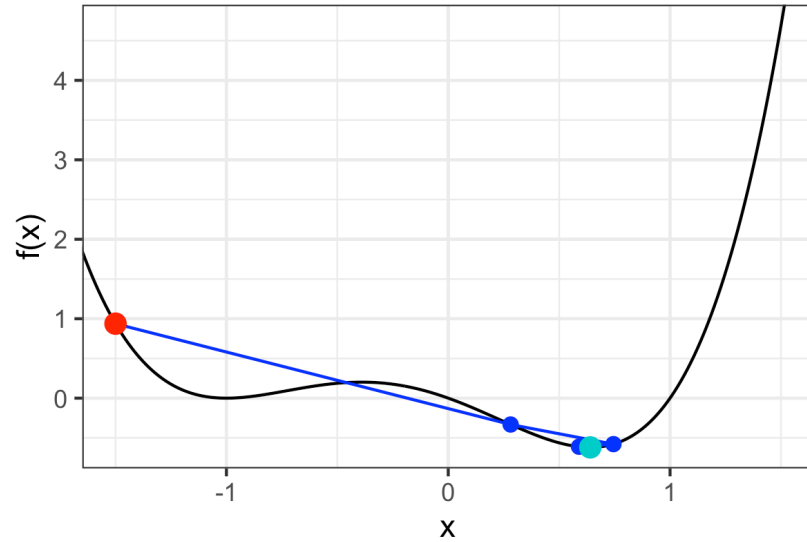
Implementation

```
1 grad_desc_1d_bt = function(x, f, grad, step, tau = 0.5, max_step = 100, max_back = 10, tol =
2   res = list(x = x, f = f(x))
3
4   for (i in seq_len(max_step)) {
5     grad_f = grad(x)
6     for (j in seq_len(max_back)) {
7       x_new = x - step * grad_f
8       f_new = f(x_new)
9       if (f_new < tail(res$f, 1))
10        break
11     step = step * tau
12   }
13
14   if (abs(x_new - x) < tol)
15     break
16
17   x = x_new
18   res$x = c(res$x, x)
19   res$f = c(res$f, f_new)
20 }
21
22 if (i == max_step)
23   warning("Failed to converge!")
```

```

1 opt = grad_desc_1d_bt(
2   -1.5, f, grad, step = 0.75, tau = 0.5
3 )
4 plot_1d_traj(c(-1.5, 1.5), f, opt)

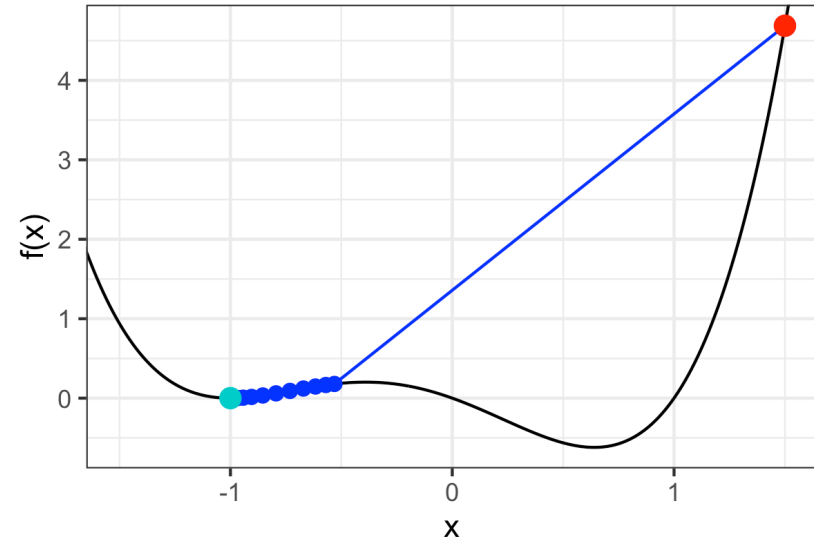
```



```

1 opt = grad_desc_1d_bt(
2   1.5, f, grad, step = 0.25, tau = 0.5
3 )
4 plot_1d_traj(c(-1.5, 1.5), f, opt)

```



A 2d function

We will be using `mk_quad()` to create quadratic functions with varying conditioning (as specified by the `epsilon` parameter).

$$f(x, y) = 0.33(x^2 + \epsilon^2 y^2)$$

$$\nabla f(x, y) = \begin{bmatrix} 0.66 x \\ 0.66 \epsilon^2 y \end{bmatrix}$$

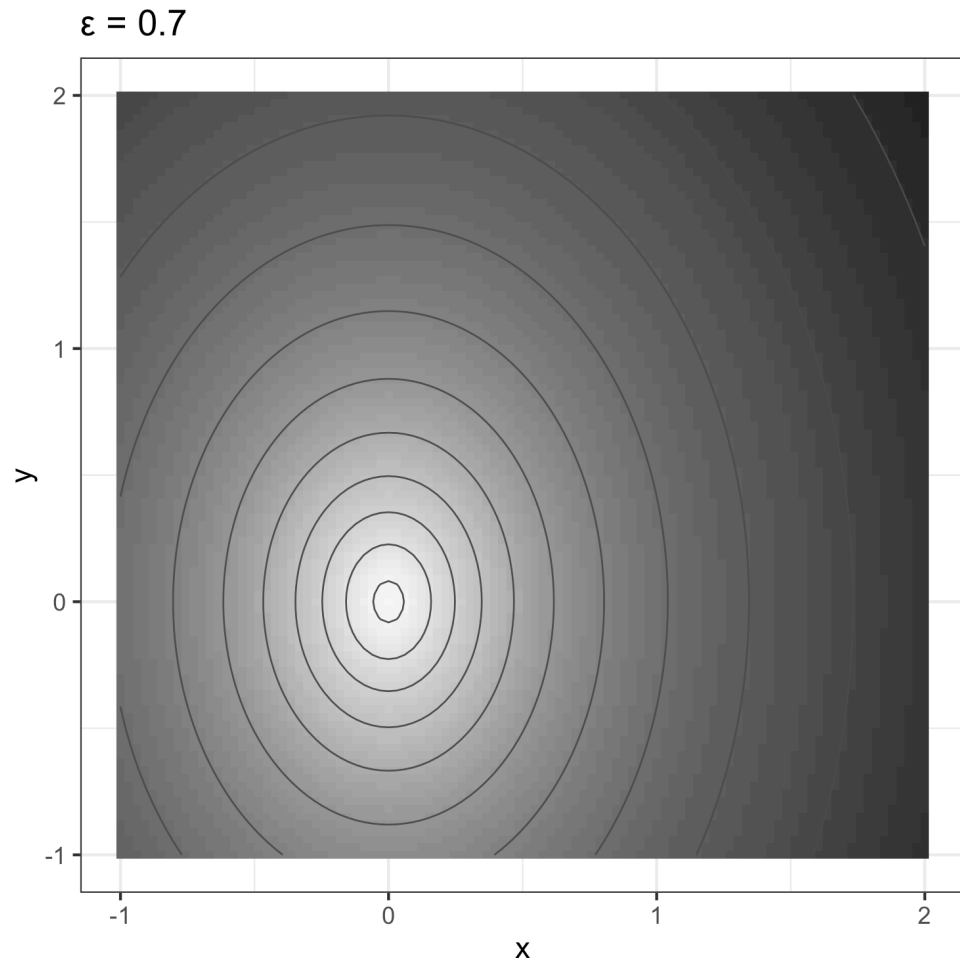
$$\nabla^2 f(x, y) = \begin{bmatrix} 0.66 & 0 \\ 0 & 0.66 \epsilon^2 \end{bmatrix}$$

Examples

```

1 q = mk_quad(0.7)
2 plot_2d_traj(
3   c(-1, 2), c(-1, 2), q$f,
4   title = "ε = 0.7"
5 )

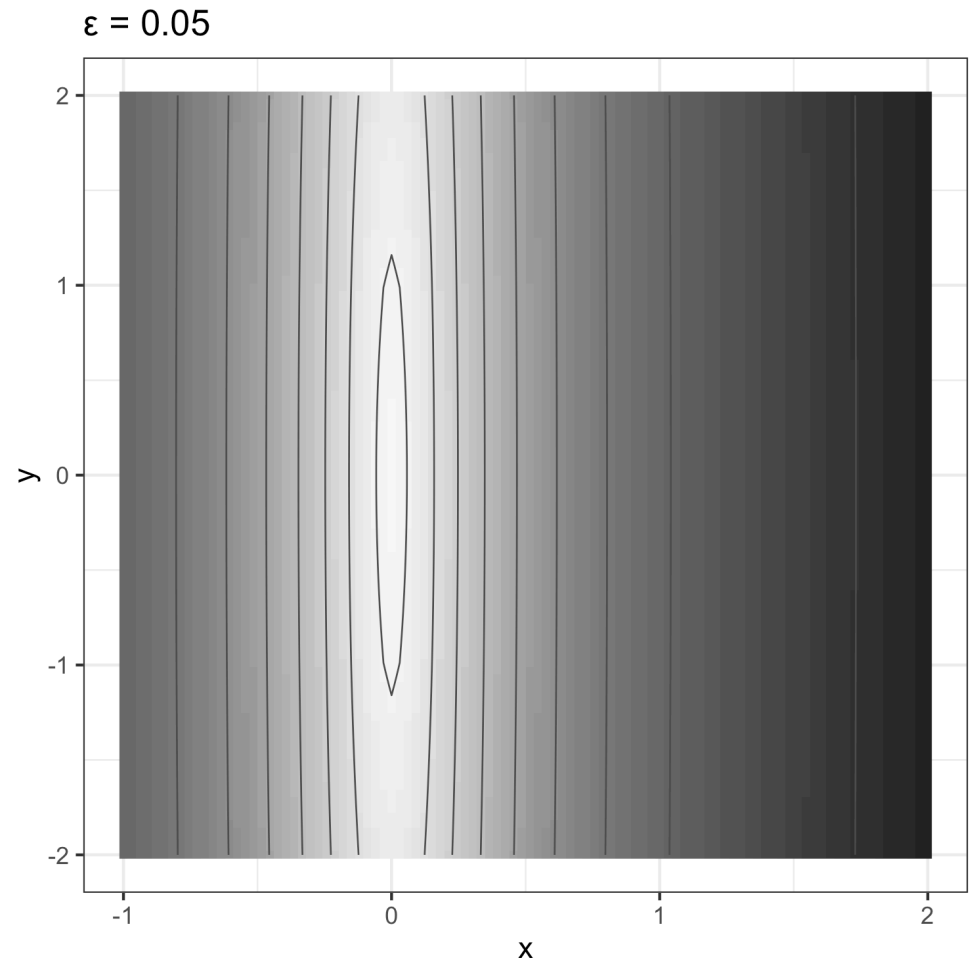
```



```

1 q = mk_quad(0.05)
2 plot_2d_traj(
3   c(-1, 2), c(-2, 2), q$f,
4   title = "ε = 0.05"
5 )

```



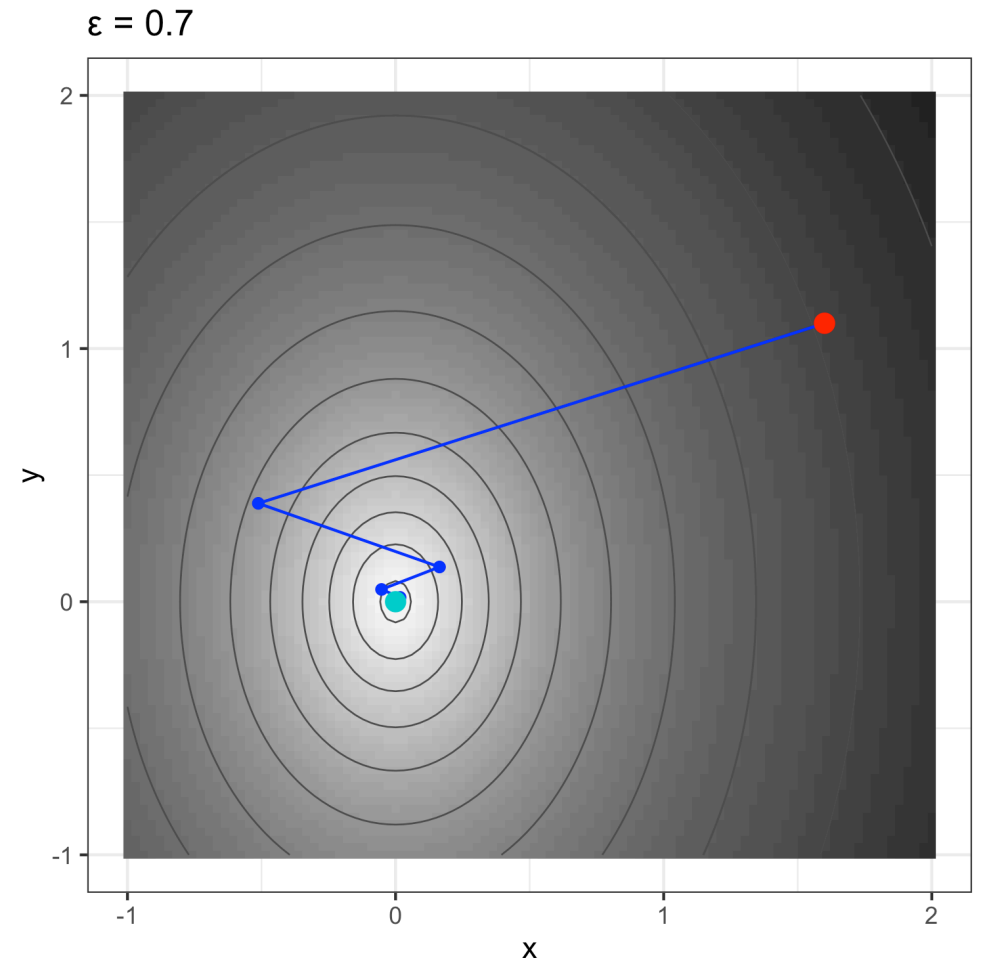
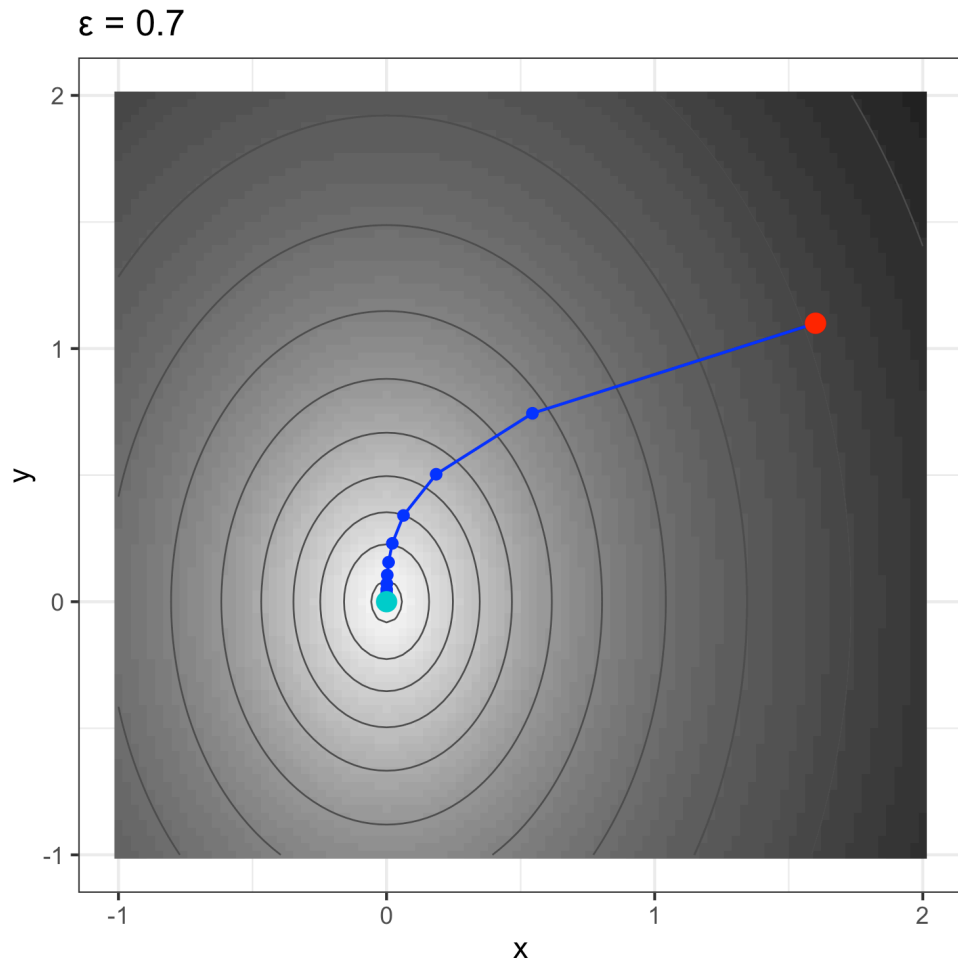
n -d gradient descent w/ backtracking

```
1 grad_desc = function(x, f, grad, step, tau = 0.5, max_step = 100, max_back = 10, tol = 1e-8)
2   res = list(x = list(x), f = f(x))
3
4   for (i in seq_len(max_step)) {
5     grad_f = grad(x)
6
7     for (j in seq_len(max_back)) {
8       x_new = x - grad_f * step
9       f_new = f(x_new)
10      if (f_new < tail(res$f, 1))
11        break
12      step = step * tau
13    }
14
15    if (sqrt(sum((x_new - x)^2)) < tol)
16      break
17
18    x = x_new
19    res$x = c(res$x, list(x))
20    res$f = c(res$f, f_new)
21  }
22
23  if (i == max_step)
```

Well conditioned cost function

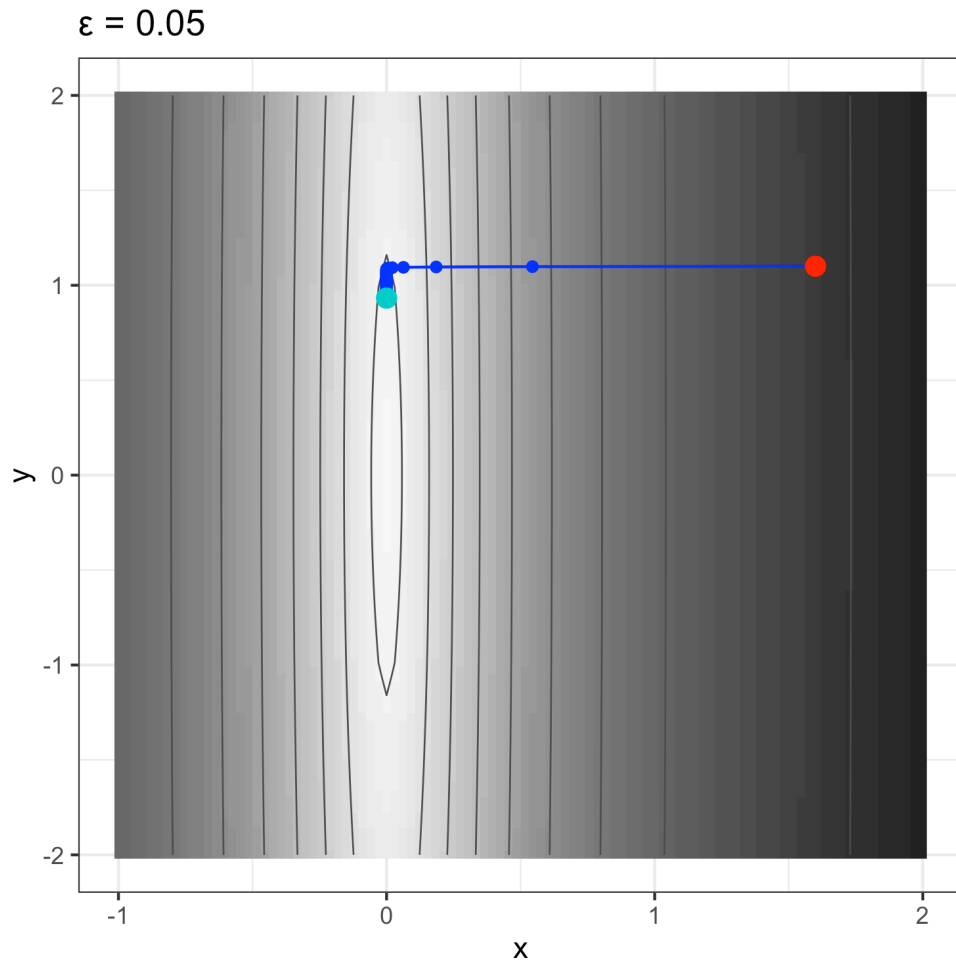
```
1 q = mk_quad(0.7)
2 opt = grad_desc(c(1.6, 1.1), q$f, q$grad, step = 1)
3 plot_2d_traj(
4   c(-1, 2), c(-1, 2), q$f, traj = opt, title = "ε = 0.7"
5 )
```

```
1 q = mk_quad(0.7)
2 opt = grad_desc(c(1.6, 1.1), q$f, q$grad, step = 2)
3 plot_2d_traj(
4   c(-1, 2), c(-1, 2), q$f, traj = opt, title = "ε = 0.7"
5 )
```

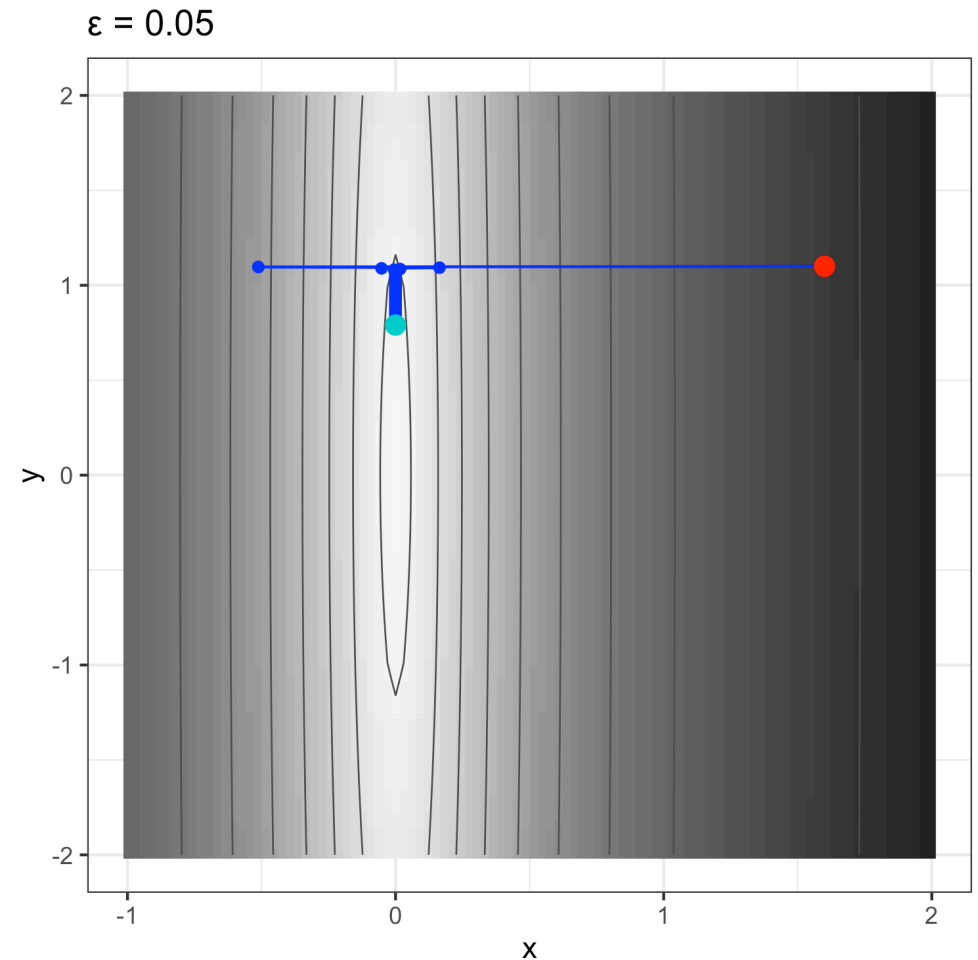


Ill-conditioned cost function

```
1 q = mk_quad(0.05)
2 opt = grad_desc(c(1.6, 1.1), q$f, q$grad, step = 1)
3 plot_2d_traj(
4   c(-1, 2), c(-2, 2), q$f, traj = opt, title = "ε = 0.05"
5 )
```



```
1 q = mk_quad(0.05)
2 opt = grad_desc(c(1.6, 1.1), q$f, q$grad, step = 2)
3 plot_2d_traj(
4   c(-1, 2), c(-2, 2), q$f, traj = opt, title = "ε = 0.05"
5 )
```



Rosenbrock's function

This is another classic ill-conditioned function commonly used to benchmark optimization algorithms. It has a narrow, curved (banana-shaped) valley - easy to find the valley, hard to follow it to the minimum

Classic definition is,

$$f(u, v) = \frac{1}{2}(1 - u)^2 + (v - u^2)^2$$

Our version is scaled and uses $u = 4x + 1$ and $v = 4y + 3$,

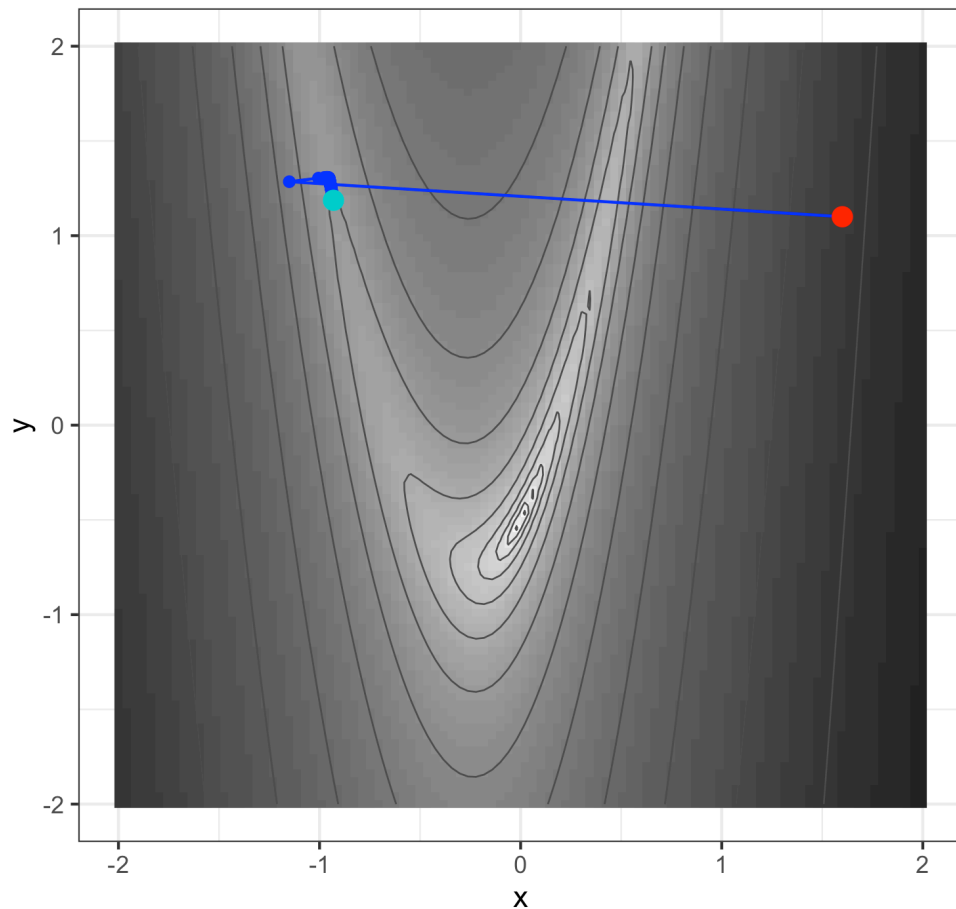
$$f(x, y) = 8x^2 + (4y + 3 - (4x + 1)^2)^2$$

The minimum is at $(u^*, v^*) = (1, 1)$, $(x^*, y^*) = (0, -\frac{1}{2})$ where $f = 0$

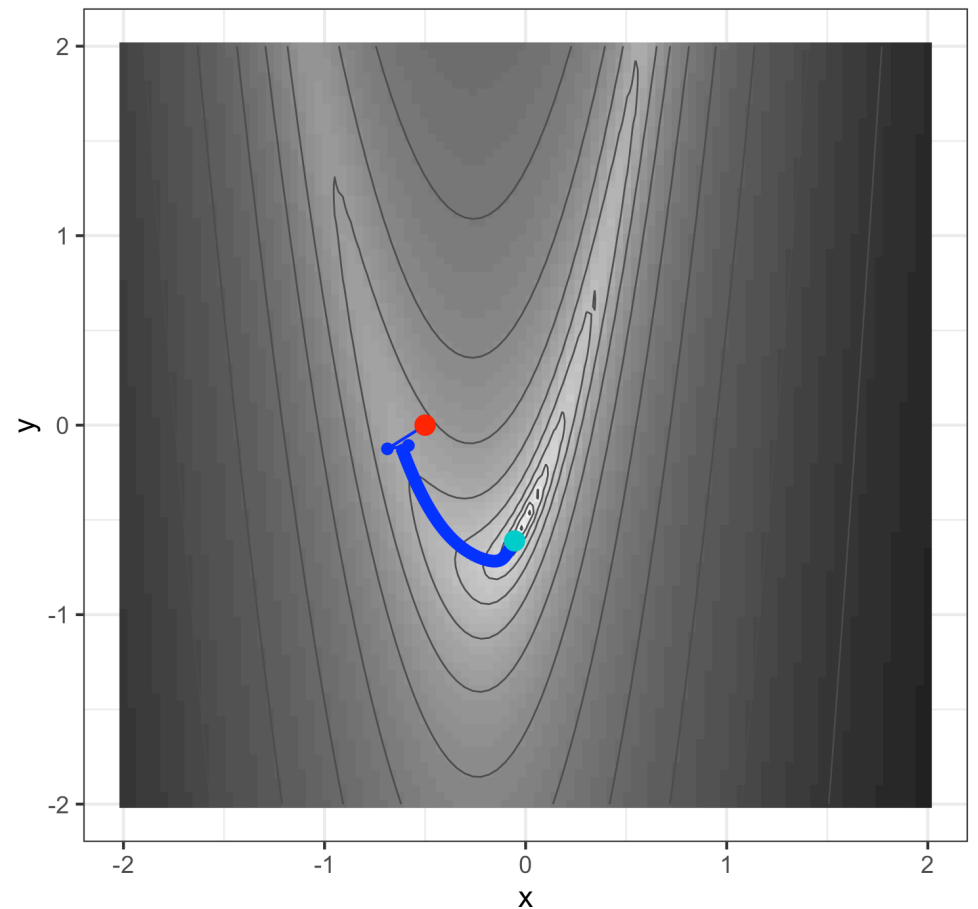
Rosenbrock function (very ill conditioned)

```
1 rb = mk_rosenbrock()
```

```
1 opt = grad_desc(c(1.6, 1.1), rb$f, rb$grad, step = 1e-6)  
2 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj = opt)
```



```
1 opt = grad_desc(c(-0.5, 0), rb$f, rb$grad, step = 1e-6)  
2 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj = opt)
```



Limitations of gradient descent

- Converges to a *local* minima - sensitive to starting location
 - Global convergence guarantees only hold for convex functions
- Requires gradient computation at every step - expensive when n is large or the gradient has no closed form
- Sensitive to the choice of learning rate - too large diverges, too small converges slowly
- Uses a scalar step size applied uniformly in all directions
 - Performs poorly on ill-conditioned problems where the optimal step varies by direction

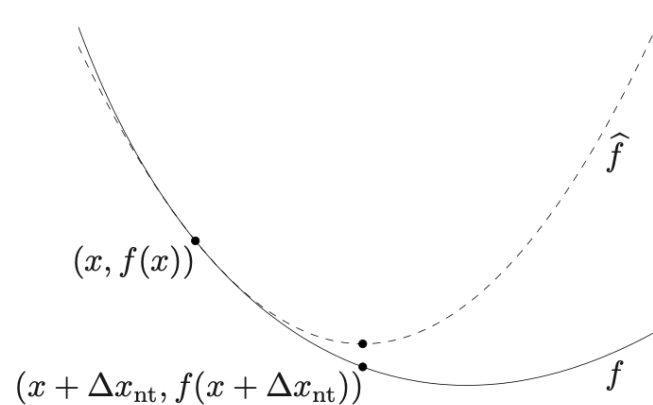
Newton's Method

Newton's Method in 1d

Let's assume we have a 1d function $f(x)$ we are trying to optimize, our current guess is x and we want to know how to generate our next step Δx .

We start by constructing the 2nd order Taylor approximation of our function at $x + \Delta x$,

$$f(x + \Delta x) \approx \hat{f}(x + \Delta x) = f(x) + \Delta x f'(x) + \frac{1}{2}(\Delta x)^2 f''(x)$$



Finding the Newton step

Our optimal step then becomes the value of Δx that minimizes the quadratic given by our Taylor approximation.

$$\frac{\partial}{\partial \Delta x} \widehat{f}(x + \Delta x) = 0$$

$$\frac{\partial}{\partial \Delta x} \left(f(x) + \Delta x f'(x) + \frac{1}{2}(\Delta x)^2 f''(x) \right) = 0$$

$$f'(x) + \Delta x f''(x) = 0$$

$$\Delta x = -\frac{f'(x)}{f''(x)}$$

this suggests an iterative update rule of

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}$$

Generalizing to nd

Based on the same argument we can see the following result for a function in $\mathbb{R}^n$,

$$f(x + \Delta x) \approx \widehat{f}(x) = f(x) + \Delta x^T \nabla f(x) + \frac{1}{2} \Delta x^T \nabla^2 f(x) \Delta x$$

then

$$\frac{\partial}{\partial \Delta x} \widehat{f}(x) = 0$$

$$\nabla f(x) + \nabla^2 f(x) \Delta x = 0$$

$$\Delta x = -(\nabla^2 f(x))^{-1} \nabla f(x)$$

where

- $\nabla f(x)$ is the $n \times 1$ gradient vector
- and $\nabla^2 f(x)$ is the $n \times n$ Hessian matrix.

based on these results our nd update rule is

$$x_{k+1} = x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k)$$

Implementation

```
1 newtons_method = function(x, f, grad, hess, max_iter = 100, tol = 1e-8) {  
2   res = list(x = list(x), f = f(x))  
3  
4   for (i in seq_len(max_iter)) {  
5     g = grad(x)  
6     H = hess(x)  
7     x_new = x - solve(H, g)  
8  
9     if (sqrt(sum((x_new - x)^2)) < tol)  
10      break  
11  
12     x = x_new  
13     res$x = c(res$x, list(x))  
14     res$f = c(res$f, f(x))  
15   }  
16  
17   res  
18 }
```

A basic example

$$f(x) = x^2$$
$$\nabla f(x) = 2x$$
$$\nabla^2 f(x) = 2$$

```
1 f = \(\x) x^2
2 grad = \(\x) 2 * x
3 hess = \(\x) matrix(2)
```

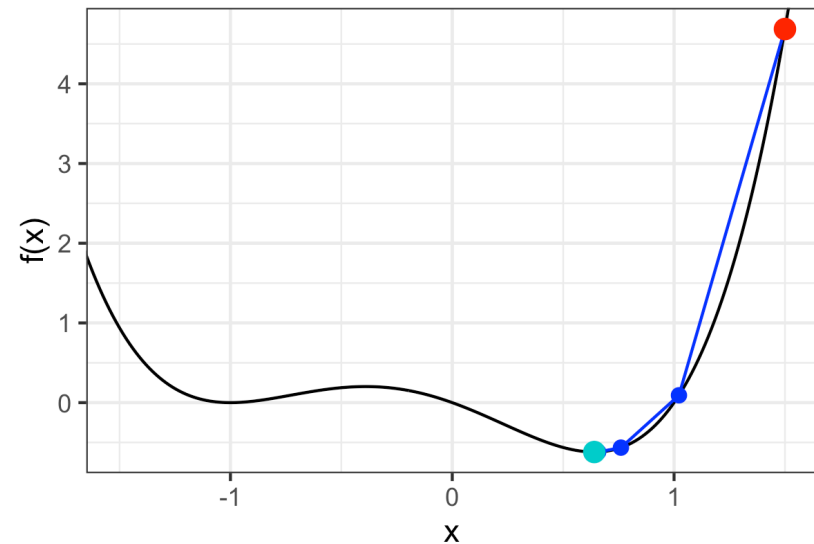
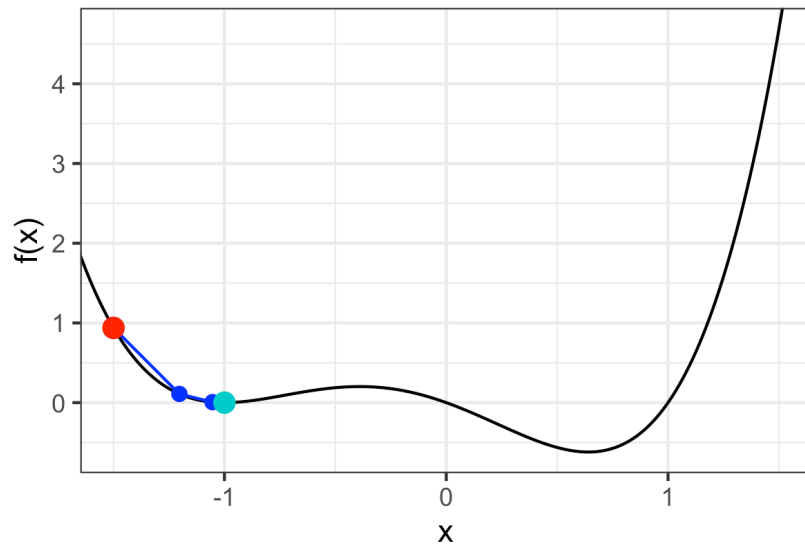
1d Quartic

$$\begin{aligned}f(x) &= x^4 + x^3 - x^2 - x \\ \nabla f(x) &= 4x^3 + 3x^2 - 2x - 1 \\ \nabla^2 f(x) &= 12x^2 + 6x - 2\end{aligned}$$

```
1 f = \(\x) x^4 + x^3 - x^2 - x
2 grad = \(\x) 4*x^3 + 3*x^2 - 2*x - 1
3 hess = \(\x) 12*x^2 + 6*x - 2
```

```
1 opt = newtons_method(-1.5, f, \(\x) matrix(g
2 plot_1d_traj(c(-1.5, 1.5), f, opt)
```

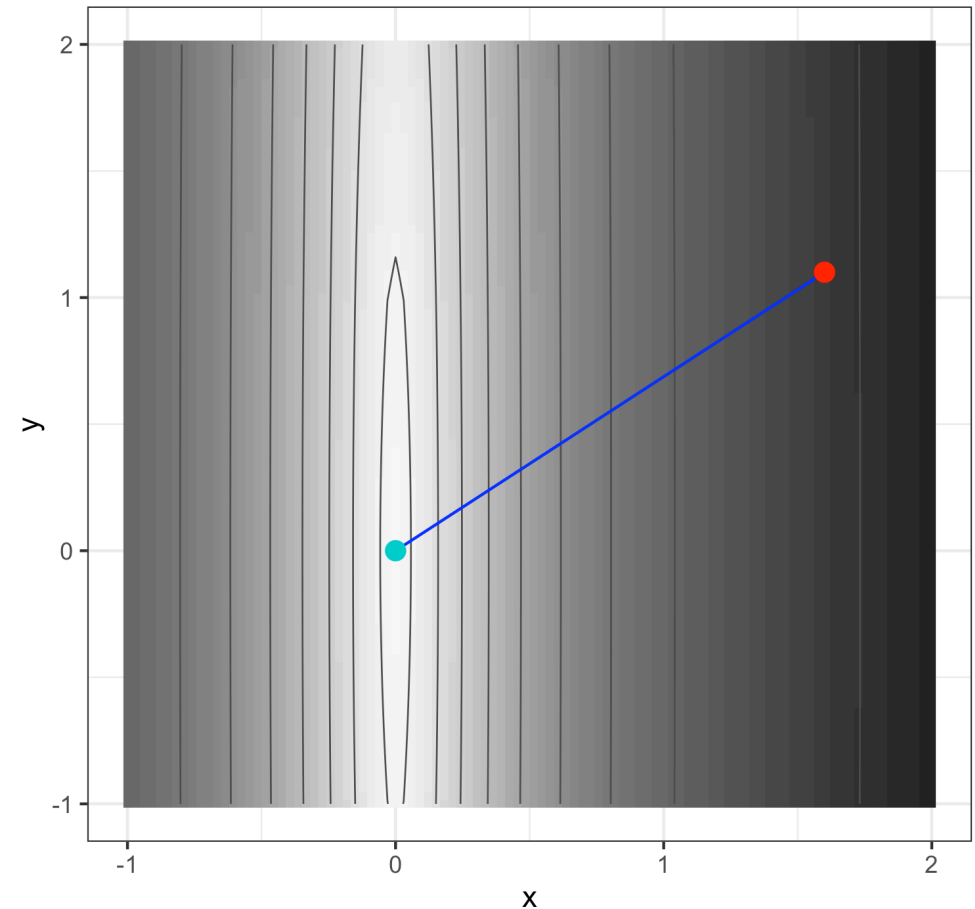
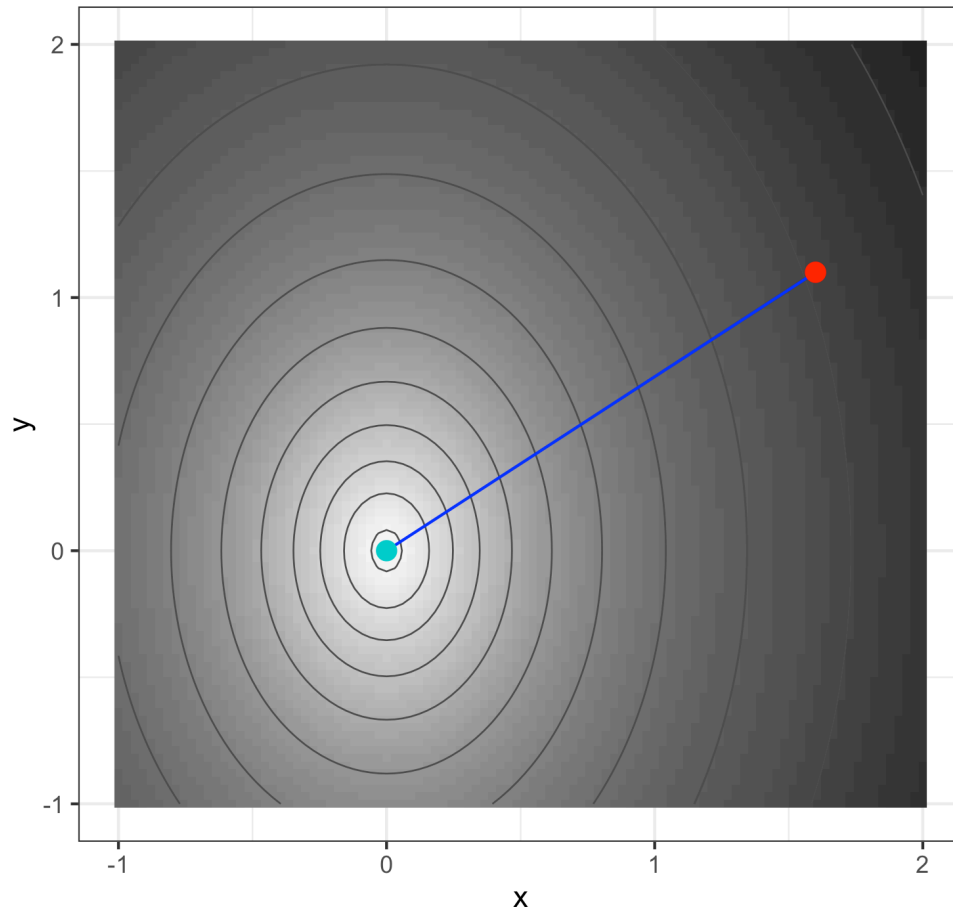
```
1 opt = newtons_method(1.5, f, \(\x) matrix(g
2 plot_1d_traj(c(-1.5, 1.5), f, opt)
```



2d quadratic cost functions

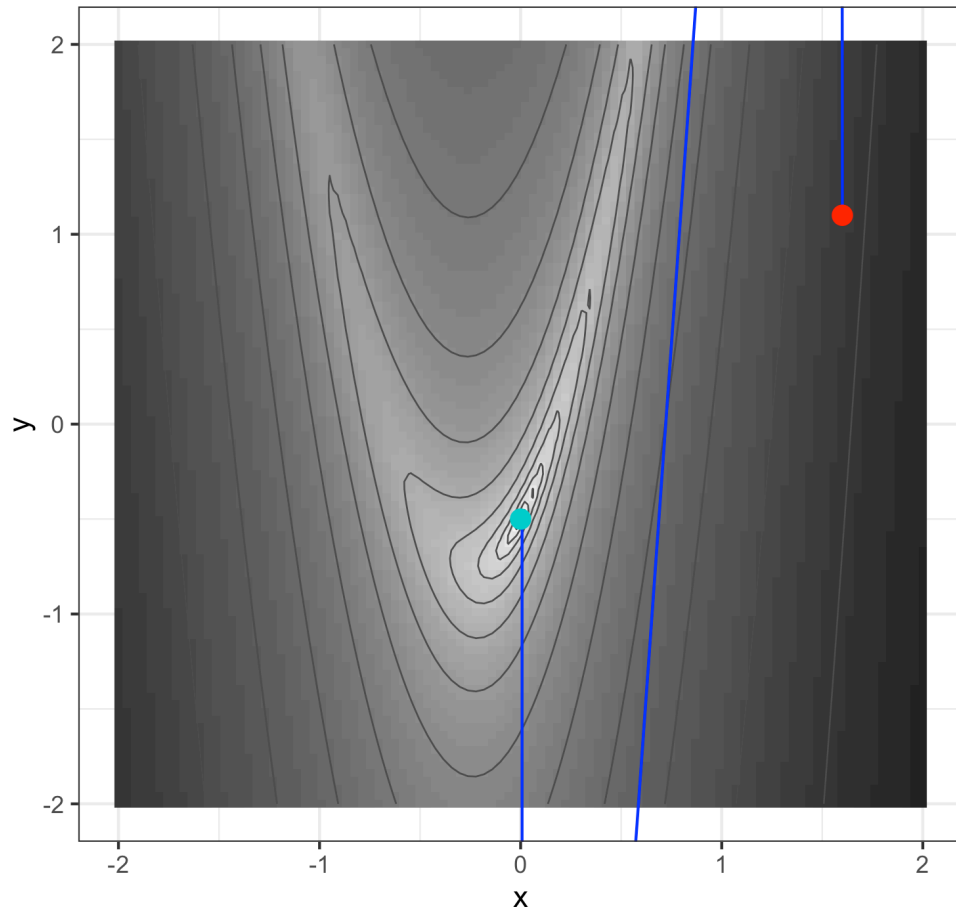
```
1 q = mk_quad(0.7)
2 opt = newtons_method(c(1.6, 1.1), q$f, q$gr)
3 plot_2d_traj(c(-1, 2), c(-1, 2), q$f, traj =
```

```
1 q = mk_quad(0.05)
2 opt = newtons_method(c(1.6, 1.1), q$f, q$gr)
3 plot_2d_traj(c(-1, 2), c(-1, 2), q$f, traj =
```

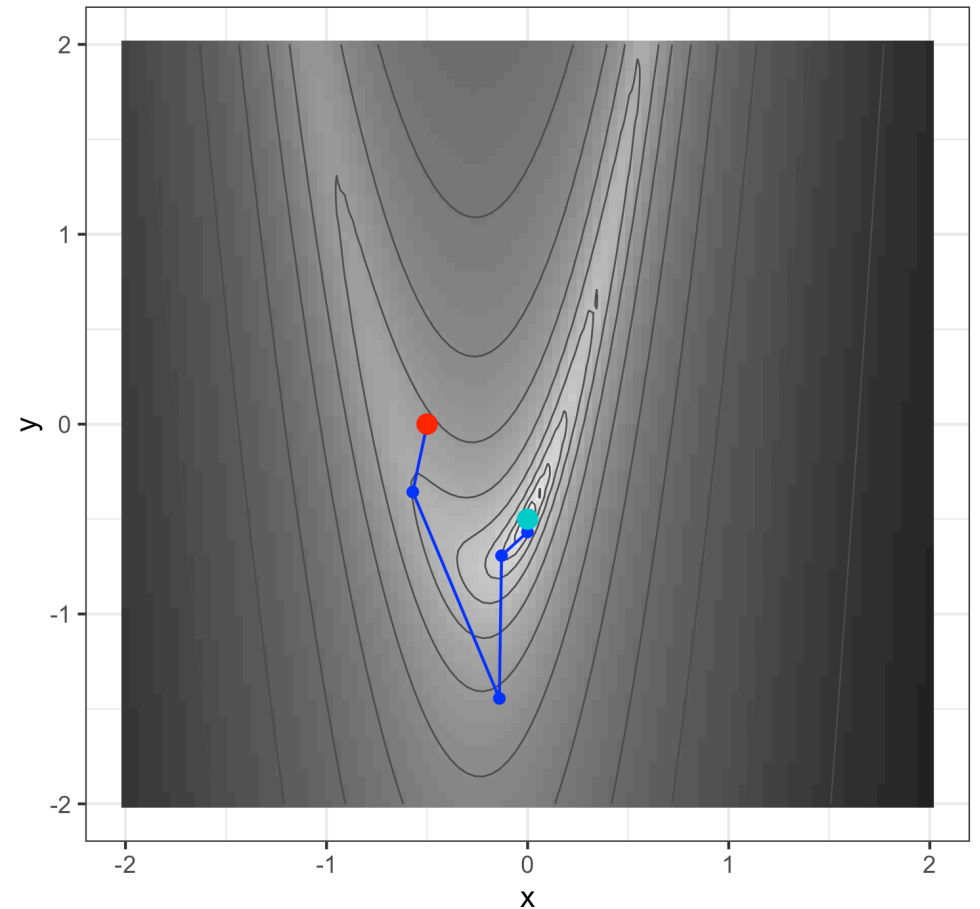


Rosenbrock function

```
1 rb = mk_rosenbrock()
2 opt = newtons_method(c(1.6, 1.1), rb$f, rb$g)
3 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj)
```



```
1 rb = mk_rosenbrock()
2 opt = newtons_method(c(-0.5, 0), rb$f, rb$g)
3 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj)
```



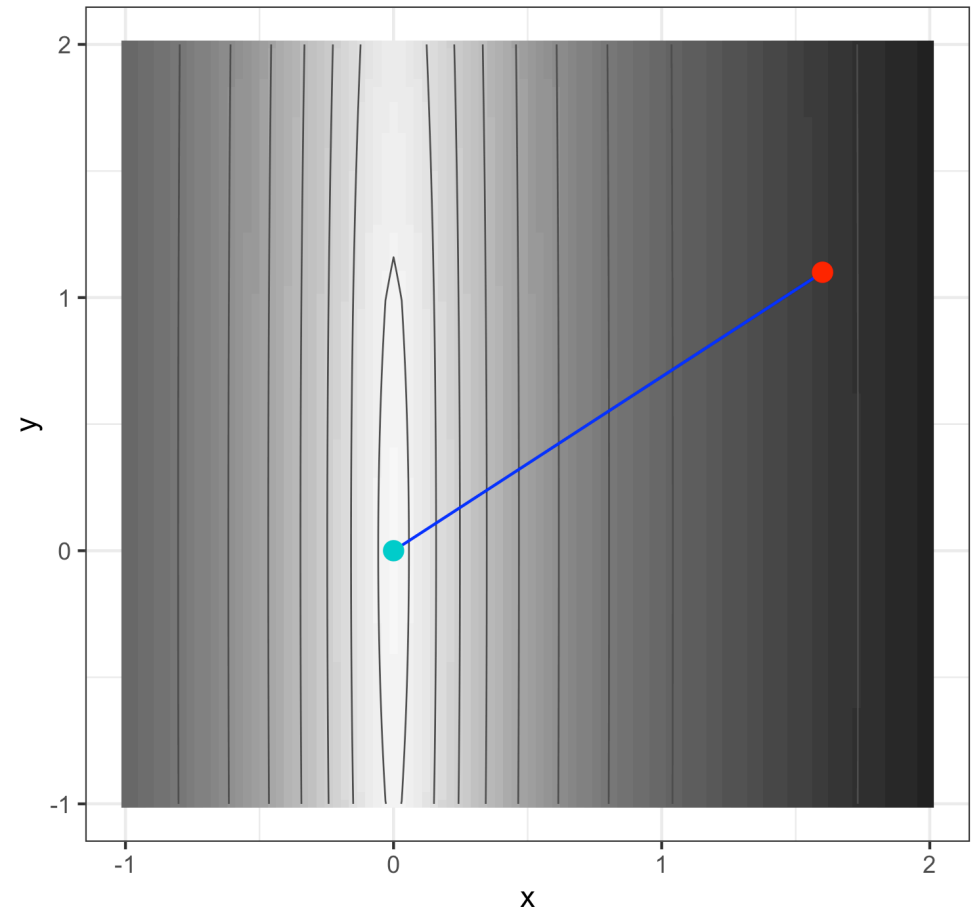
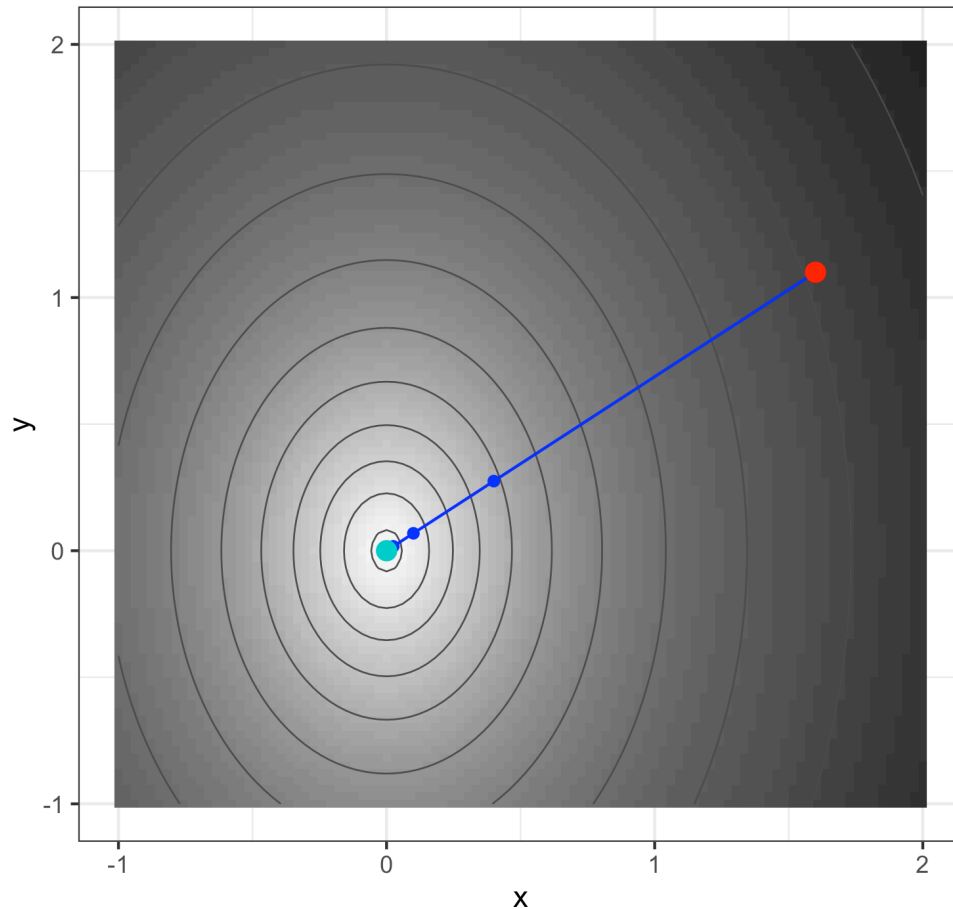
Damped / backtracking implementation

```
1 newtons_method_damped = function(  
2   x, f, grad, hess, max_iter = 100, max_back = 10, tol = 1e-8,  
3   alpha = 0.5, beta = 0.75  
4 ) {  
5   res = list(x = list(x), f = f(x))  
6  
7   for (i in seq_len(max_iter)) {  
8     grad_f = grad(x)  
9     step = -solve(hess(x), grad_f)  
10    t = 1  
11    for (j in seq_len(max_back)) {  
12      if (f(x + t * step) < f(x) + alpha * t * sum(grad_f * step))  
13        break  
14      t = t * beta  
15    }  
16  
17    x_new = x + t * step  
18  
19    if (sqrt(sum((x_new - x)^2)) < tol)  
20      break  
21  
22    x = x_new  
23    res$x = c(res$x, list(x))
```

2d quadratic cost function

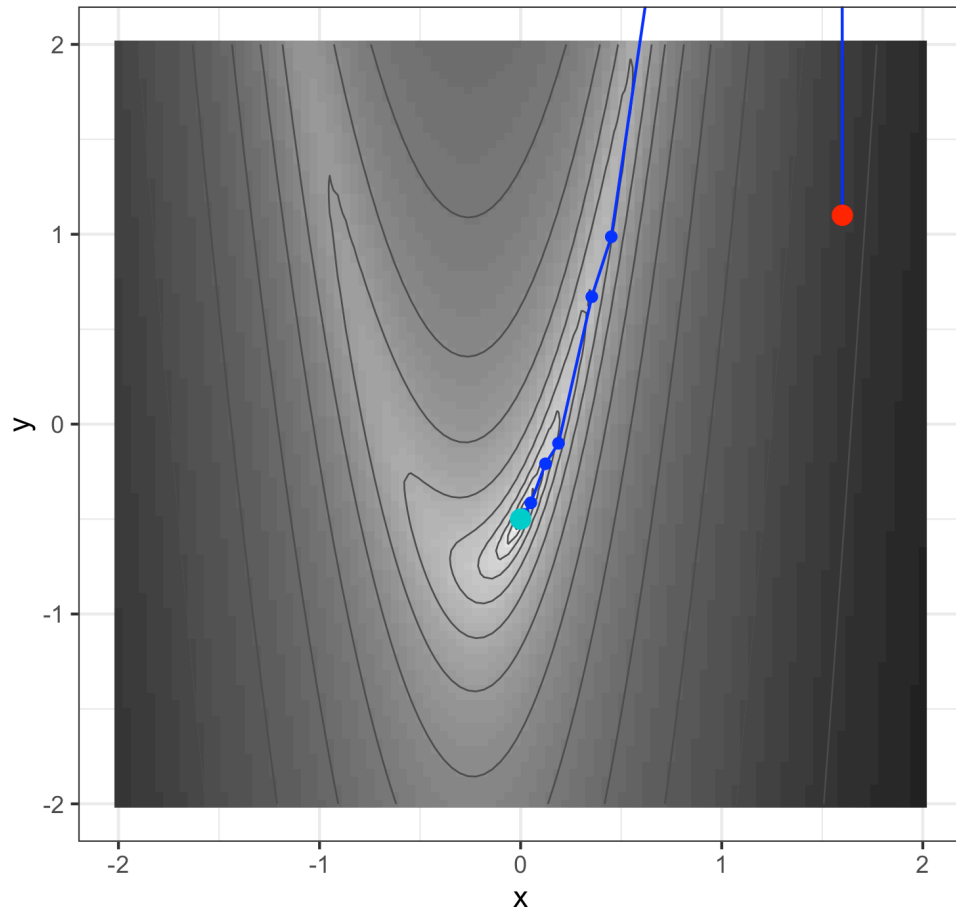
```
1 q = mk_quad(0.7)
2 opt = newtons_method_damped(c(1.6, 1.1), q$f)
3 plot_2d_traj(c(-1, 2), c(-1, 2), q$f, traj =
```

```
1 q = mk_quad(0.05)
2 opt = newtons_method_damped(c(1.6, 1.1), q$f)
3 plot_2d_traj(c(-1, 2), c(-1, 2), q$f, traj =
```

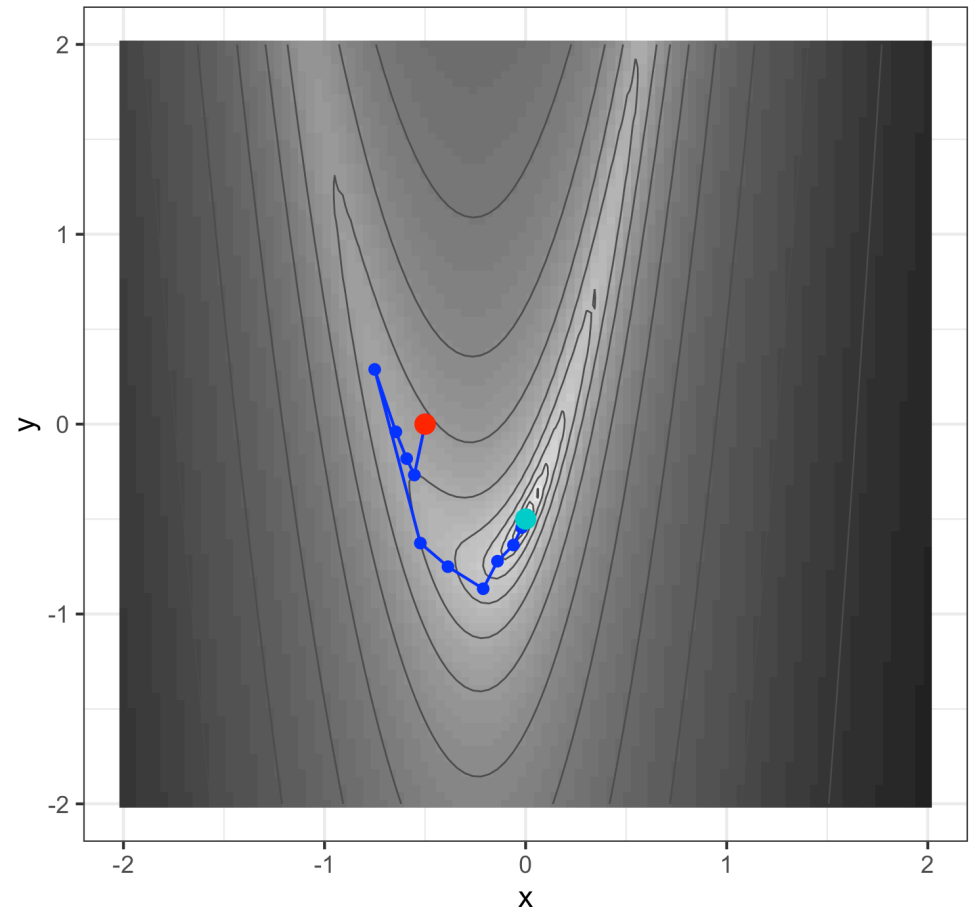


Rosenbrock function

```
1 rb = mk_rosenbrock()
2 opt = newtons_method_damped(c(1.6, 1.1), rb)
3 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj)
```



```
1 rb = mk_rosenbrock()
2 opt = newtons_method_damped(c(-0.5, 0), rb)
3 plot_2d_traj(c(-2, 2), c(-2, 2), rb$f, traj)
```



Limitations of Newton's Method

- Requires both the gradient *and* Hessian - second derivatives may be unavailable or expensive to compute
- Storing the $n \times n$ Hessian costs $O(n^2)$ memory and inverting it costs $O(n^3)$ per step - impractical for large n
- Can diverge or behave poorly if the Hessian is indefinite (not positive definite) away from the minimum
- Like GD, finds *local* minima and is sensitive to starting location
- Damping (backtracking) helps with robustness but adds cost and complexity

Method Comparison

Method	Convergence	Cost per step	Needs Hessian?	Notes
Gradient Descent	Linear	$O(n)$	No	Simple but slow on ill-conditioned problems
Newton's Method	Quadratic	$O(n^3)$	Yes	Fast near minimum; expensive for large n

Convergence rates matter in practice:

- *Linear* convergence means the error reduces by a constant factor each step - fine for well-conditioned problems, slow otherwise
- *Quadratic* convergence means the number of correct digits roughly doubles each step - Newton's method achieves this near the minimum

R's `optim()`

R's `optim()`

R's `optim()` function (from the `stats` package) provides a set of general-purpose optimization methods:

Method	Description	Gradient	Bounds
"Nelder-Mead"	Nelder-Mead simplex (default)	$\times$	$\times$
"BFGS"	Quasi-Newton (Broyden-Fletcher-Goldfarb-Shanno)	Optional	$\times$
"L-BFGS-B"	Limited-memory BFGS with box constraints	Optional	$\checkmark$
"CG"	Conjugate gradient (Fletcher-Reeves, Polak-Ribiere, or Beale-Sorenson)	Optional	$\times$
"SANN"	Simulated annealing (stochastic global optimization)	$\times$	$\times$

- "BFGS" iteratively approximates the inverse Hessian - no Hessian needed, gradient optional
- "L-BFGS-B" stores only the last m updates ($m \approx 5-20$), reducing memory from $O(n^2)$ to $O(mn)$
- The `optimx` package provides access to additional methods beyond those in `optim()`

Nelder-Mead

This is a gradient free method that uses a series of simplexes which are used to iteratively bracket the minimum.

optim() interface

```
1 optim(par, fn, gr = NULL, method = "Nelder-Mead",  
2       lower = -Inf, upper = Inf, control = list(), hessian = FALSE)
```

Key arguments:

- `par` - initial parameter values (numeric vector)
- `fn` - function to minimize (must take a vector and return a scalar)
- `gr` - gradient function (optional; if `NULL`, finite differences are used)
- `method` - optimization method
- `lower`, `upper` - bounds (only for "L-BFGS-B")
- `control` - list of control parameters (e.g. `maxit`, `reltol`, `fnscale`)
- `hessian` - if `TRUE`, return a numerically approximated Hessian

Example - Well-conditioned quadratic

```
1 q = mk_quad(0.7)
```

```
1 optim(  
2   c(1.6, 1.1), q$f, q$grad,  
3   method = "BFGS"  
4 )
```

```
$par  
[1] 5.862023e-11 -4.031139e-11
```

```
$value  
[1] 1.396753e-21
```

```
$counts  
function gradient  
      18      15
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
1 optim(  
2   c(1.6, 1.1), q$f,  
3   method = "BFGS"  
4 )
```

```
$par  
[1] 5.862023e-11 -4.031139e-11
```

```
$value  
[1] 1.396753e-21
```

```
$counts  
function gradient  
      18      15
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

Example - Rosenbrock

```
1 rb = mk_rosenbrock()
```

```
1 optim(  
2   c(1.6, 1.1), rb$f, rb$grad,  
3   method = "BFGS"  
4 )
```

```
$par  
[1] 1.10249e-12 -5.00000e-01
```

```
$value  
[1] 1.049227e-23
```

```
$counts  
function gradient  
      49      20
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```
1 optim(  
2   c(1.6, 1.1), rb$f, rb$grad,  
3   method = "Nelder-Mead"  
4 )
```

```
$par  
[1] 0.0008425688 -0.4982432286
```

```
$value  
[1] 5.7551e-06
```

```
$counts  
function gradient  
      119      NA
```

```
$convergence  
[1] 0
```

```
$message  
NULL
```

```

1 optim(
2   c(1.6, 1.1), rb$f, rb$grad,
3   method = "L-BFGS-B"
4 )

```

```

$par
[1] 1.543156e-09 -5.000000e-01

```

```

$value
[1] 3.58706e-17

```

```

$counts
function gradient
      26      26

```

```

$convergence
[1] 0

```

```

$message
[1] "CONVERGENCE: REL_REDUCTION_OF_F <= FACTR*EP

```

```

1 optim(
2   c(1.6, 1.1), rb$f, rb$grad,
3   method = "CG"
4 )

```

```

$par
[1] 0.0009579996 -0.4978521443

```

```

$value
[1] 7.220552e-06

```

```

$counts
function gradient
      356      101

```

```

$convergence
[1] 1

```

```

$message
[1] NULL

```

Comparing methods - fixed start

Implementation

Results

```
1 cost_funcs = list(  
2   "Well-cond quad" = mk_quad(0.7),  
3   "Ill-cond quad" = mk_quad(0.02),  
4   "Rosenbrock" = mk_rosenbrock()  
5 )  
6  
7 results_fixed = bench::press(  
8   cost_func = names(cost_funcs),  
9   {  
10     fns = cost_funcs[[cost_func]]  
11     bench::mark(  
12       "Nelder-Mead" = optim(c(1.6, 1.1), fns$f, fns$grad, method = "Nelder-Mead"),  
13       "BFGS"       = optim(c(1.6, 1.1), fns$f, fns$grad, method = "BFGS"),  
14       "CG"         = optim(c(1.6, 1.1), fns$f, fns$grad, method = "CG"),  
15       "L-BFGS-B"   = optim(c(1.6, 1.1), fns$f, fns$grad, method = "L-BFGS-B"),  
16       check = FALSE  
17     )  
18   }  
19 )
```

Comparing methods - random start

Implementation

Results

```
1 results_random = bench::press(  
2   cost_func = names(cost_funcs),  
3   x0 = runif(200, -2, 2) |> matrix(ncol=2) |> apply(1,c, simplify = FALSE),  
4   {  
5     fns = cost_funcs[[cost_func]]  
6     x0 = unlist(x0)  
7     bench::mark(  
8       "Nelder-Mead" = optim(x0, fns$f, fns$grad, method = "Nelder-Mead"),  
9       "BFGS"        = optim(x0, fns$f, fns$grad, method = "BFGS"),  
10      "CG"          = optim(x0, fns$f, fns$grad, method = "CG"),  
11      "L-BFGS-B"    = optim(x0, fns$f, fns$grad, method = "L-BFGS-B"),  
12      check = FALSE,  
13      iterations = 1  
14    )  
15  }  
16 ) |>  
17 select(-x0)
```

Some general advice

- Having access to the gradient is almost always helpful / necessary
- Having access to the Hessian can be helpful, but usually does not significantly improve things
- The curse of dimensionality is real
 - Be careful with tolerance settings
- In general, **BFGS** or **L-BFGS** should be a good place to start for most problems (either well- or ill-conditioned)